UNIT 1 · TOPIC 1.5

Leveraging AI in Cyber Defense

Assignment

Instructions: 20 practice questions + 4 coding assignment(s), 39 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

1.5.a · AI reviewing security configurations

1. A config file contains 'PermitRootLogin yes'. What is the DIRECT security risk? (1 pt)

- ○ A. The SSH port becomes publicly visible
- ○ B. An attacker who cracks the root password has immediate full system access
- ○ C. The system disables all password authentication
- ○ D. TLS encryption is bypassed

2. Which of the following settings in an SSH config file would an AI auditor flag as insecure? Select ALL that apply. (3 pts)

- □ A. PasswordAuthentication yes
- □ B. Port 22
- □ C. PermitRootLogin yes
- □ D. X11Forwarding yes

(Select ALL that apply.)

3. Why does an AI config auditor emit a FINDING rather than automatically changing the insecure setting? (1 pt)

- ○ A. AI tools cannot read configuration files
- ○ B. Configuration files are encrypted
- ○ C. The setting may have a compensating control or documented exception that the human reviewer must evaluate
- ○ D. Audit findings are only used for compliance reports, not remediation

4. A config line reads 'port 23'. Why does an AI auditor flag this? (1 pt)

- ○ A. Port 23 is used by many attack tools, so seeing it means the server is compromised
- ○ B. Port 23 is Telnet, which transmits credentials in cleartext
- ○ C. Any port below 1024 requires root access, which is insecure
- ○ D. Port 23 is the default port for unencrypted HTTP

5. Name the security lifecycle stage where config review runs inside a CI/CD pipeline to block insecure settings BEFORE they reach production. (1 pt)

1.5.b · AI analyzing code for vulnerabilities

6. Why is `eval(user_input)` flagged as dangerous by a static code scanner? (1 pt)

- ○ A. eval() is slower than normal code, which lets attackers cause a denial of service
- ○ B. eval() executes an arbitrary string as Python code, letting an attacker run any code on the server
- ○ C. eval() prints its input to the server logs, leaking user data
- ○ D. eval() turns off Python's error handling for the rest of the program

7. Which of the following Python patterns would an AI code scanner flag as causing OS command injection risk? Select ALL that apply. (2 pts)

- □ A. os.system(cmd)
- □ B. subprocess.run(args, shell=False)
- □ C. subprocess.run(cmd, shell=True)
- □ D. os.path.join(base, filename)

(Select ALL that apply.)

8. A developer stores `password = 'Admin@2024'` directly in source code. What is the PRIMARY security risk? (1 pt)

- ○ A. The password is too short to meet the 12-character minimum
- ○ B. Anyone who runs the program can see the password on screen
- ○ C. The password will expire and break the program in 90 days
- ○ D. The password is visible to anyone with access to the source code or version history, making it a leaked credential

9. Why is `pickle.loads(data)` dangerous when `data` comes from an untrusted source? (1 pt)

- ○ A. pickle is slower than JSON, so large untrusted inputs can freeze the server
- ○ B. Deserialization of a maliciously crafted pickle can execute arbitrary code without any additional exploit
- ○ C. pickle stores data unencrypted, so attackers can read it in transit
- ○ D. pickle files can contain viruses that antivirus software misses

10. Name the key difference between STATIC analysis and DYNAMIC analysis of source code. (1 pt)

1.5.c · AI suggesting detection rules

11. A detection rule fires when the same source IP generates 5 failed logins within 60 seconds. What is the name for the numeric limit (5) in this rule? (1 pt)

- ○ A. Signature
- ○ B. Baseline
- ○ C. Threshold
- ○ D. Precision

12. An analyst sets the brute-force threshold to 1 failed login. What is the MOST LIKELY consequence? (1 pt)

- ○ A. Attackers will be detected faster, with no downside
- ○ B. Brute-force attempts will be blocked automatically
- ○ C. Alert fatigue: analysts are overwhelmed by normal mistyped passwords
- ○ D. The rule will stop firing because one failure is below the noise floor

13. Which of the following are components of a well-formed detection rule? Select ALL that apply. (3 pts)

□ A. A condition (what pattern to match)

□ B. A threshold (how many events)

□ C. An action (what to do when triggered)

□ D. A list of attacker names

(Select ALL that apply.)

14. What does SIEM stand for, and what is its primary function in a SOC? (1 pt)

15. An AI tool proposes a new detection rule based on historical log analysis. What should happen BEFORE the rule is deployed to production? (1 pt)

○ A. Deploy it immediately, because the AI learned from the organization's own logs

○ B. Deploy it in blocking mode first, then review whatever it blocks

○ C. Replace the old rules with it, since AI rules are more current

○ D. A human analyst reviews it for precision, context, and compliance

1.5.d · Faster, more accurate threat detection and response

16. MTTR stands for Mean Time to Respond. Which formula correctly defines it for a set of N incidents? (1 pt)

○ A. The sum of all response times divided by the number of incidents

○ B. The time the fastest incident took to resolve

○ C. The number of auto-resolved incidents divided by total incidents

○ D. The longest response time in the dataset

17. Which type of incident is MOST appropriate for automated (no-human) resolution? (1 pt)

○ A. A novel ransomware variant with no known decryption key

○ B. A high-confidence known-malicious IP blocked at the firewall with a reversible action

○ C. An alert involving a critical business partner's IP address

○ D. An incident requiring communication with law enforcement

18. AI-assisted triage scoring considers which of the following signals? Select ALL that apply. (3 pts)

□ A. Type of event (e.g., ransomware behavior vs. single failed login)

□ B. Criticality of the affected asset (e.g., domain controller vs. developer laptop)

□ C. The analyst's personal preference for ticket order

□ D. Historical false-positive rate for similar alert types

(Select ALL that apply.)

19. Name ONE sub-component of MTTR that AI improves by continuously monitoring telemetry without attention lapses. (1 pt)

20. A SOC processes incidents in arrival order instead of severity order. What is the PRIMARY risk? (1 pt)

○ A. Analysts will spend more time on each incident

- ○ B. Low-severity incidents will never be closed
- ○ C. Critical incidents may wait behind low-severity noise, giving attackers more dwell time
- ○ D. Auto-resolution rules will stop running

Coding Assignments (Python 3)

1.5.a · AI reviewing security configurations

3 pts · auto-graded on the site

Coding problem 1.5.a: AI reviewing security configurations

Skill: Mitigate Risk, Collaborate · **Maps to:** 1.5.A · **Language:** Python 3 (pytest)

Problem

Build the rule-engine core of an AI-assisted configuration auditor. Given a list of configuration file lines (as a real tool would read from `/etc/ssh/sshd_config` or a service config), return the sorted set of security finding tags for every insecure directive detected. This mirrors exactly what tools like Lynis and OpenSCAP do at their rule-evaluation layer.

Function contract

Implement `solve(config_lines: list[str]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return a sorted, de-duplicated list of finding tag strings.

Input

A `list[str]`, each element one line from a config file. Lines may be blank, start with # (comments), or contain leading/trailing whitespace. Matching is case-insensitive.

Example input:

```
[
    "# sshd_config for web01",
    "Port 22",
    "PermitRootLogin yes",
    "PasswordAuthentication yes",
    "X11Forwarding yes",
    "",
]
```

Output

A sorted `list[str]` of finding tags. Tags are drawn from:

- `"insecure:PermitRootLogin"`
- `"insecure:PasswordAuthentication"`
- `"insecure:Port23"`
- `"insecure:SSLDisabled"`
- `"insecure:X11Forwarding"`
- `"insecure:PermitEmptyPasswords"`

Examples

```
["PermitRootLogin yes", "Port 22"]
    -> ["insecure:PermitRootLogin"]

["PasswordAuthentication yes", "X11Forwarding yes", "PermitRootLogin yes"]
    -> ["insecure:PasswordAuthentication", "insecure:PermitRootLogin", "insecure:X11Forwarding"]

["port 23", "ssl false"]
    -> ["insecure:Port23", "insecure:SSLDisabled"]

["# all defaults", "Port 22", "PermitRootLogin no", ""]
    -> []
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.5.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(config_lines: List[str]) -> List[str]:
    """Return sorted, de-duplicated security finding tags for insecure config directives.

    Args:
        config_lines: list of strings, each one line of a configuration file.
    Returns:
        Sorted list of finding tags such as "insecure:PermitRootLogin".
        See interface.md for the full rule set.
    """
    # TODO: iterate config_lines (case-insensitive, strip whitespace, skip blank/#-comment lines)
    #       apply each rule from interface.md, collect matching tags, return sorted+deduped.
    raise NotImplementedError("implement solve() — see prompt.md")
```

1.5.b · AI analyzing code for vulnerabilities

3 pts · auto-graded on the site

Coding problem 1.5.b: AI analyzing code for vulnerabilities

Skill: Detect Attacks, Collaborate · **Maps to:** 1.5.A · **Language:** Python 3 (pytest)

Problem

Build the pattern-matching core of an AI code security scanner. Given a string of Python source code, return a sorted list of vulnerability tags for every dangerous pattern found. This is the rule-evaluation layer that tools like Bandit and Semgrep implement under the hood.

Function contract

Implement `solve(code: str) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return a sorted, de-duplicated list of finding tag strings.

Input

A single string, the entire source code to scan. May be multi-line or empty.

Output

A sorted `list[str]` of vulnerability tags drawn from:

- `"vuln:eval"`
- `"vuln:hardcoded_password"`
- `"vuln:md5"`
- `"vuln:os_system"`
- `"vuln:pickle_loads"`
- `"vuln:subprocess_shell"`

Examples

```
'result = eval(user_input)'
    -> ["vuln:eval"]

'os.system("ls " + path)\nsubprocess.run(cmd, shell=True)'
    -> ["vuln:os_system", "vuln:subprocess_shell"]

'import hashlib\nhashlib.md5(data)\npickle.loads(blob)'
    -> ["vuln:md5", "vuln:pickle_loads"]

'x = 1 + 2'
    -> []
```

Constraints

- Deterministic, offline, standard library only.
- `password` = matching is case-insensitive; all other patterns are case-sensitive.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.5.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(code: str) -> List[str]:
    """Return sorted, de-duplicated vulnerability tags for dangerous patterns in Python source code.

    Args:
        code: a string of Python source code to scan.
    Returns:
        Sorted list of finding tags such as "vuln:eval", "vuln:md5", etc.
        See interface.md for the full pattern rule set.
    """
```

```
    # TODO: search `code` for each dangerous pattern from interface.md.
    #       Note: "password =" matching is case-insensitive; others are case-sensitive.
    #       Return a sorted, de-duplicated list of finding tags.
    raise NotImplementedError("implement solve() — see prompt.md")
```

1.5.c · AI suggesting detection rules

3 pts · auto-graded on the site

Coding problem 1.5.c: Generate detection rules from a login log

Skill: Detect Attacks · **Maps to:** 1.5.A · **Language:** Python 3 (pytest)

Problem

A SOC receives a stream of authentication log events. Your task is to implement a brute-force detection engine: scan the log for source IPs that generate 5 or more `login_fail` events within any 60-second window, and emit a structured detection-rule string for each flagged IP.

This mirrors how an AI-assisted SIEM generates candidate detection rules from raw log data.

Function contract

Implement `solve(logs: list[dict]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`logs`, a list of dicts, each with:

- `ts` (int): Unix timestamp in seconds
- `src` (str): source IP address
- `event` (str): event type; only `"login_fail"` counts toward brute-force detection
- ok (bool): True = success, False = failure

Output

A **sorted** list of detection-rule strings. Format for each triggered IP:

```
ALERT brute_force src=<IP> count=<N> window=60s
```

where <N> is the maximum number of `login_fail` events from that IP found within any contiguous 60-second window (i.e., events where `max_ts - min_ts <= 59`). Rules are sorted lexicographically. At most one rule per IP.

Examples

```
logs = [
    {"ts": 0,  "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 10, "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 20, "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 30, "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 40, "src": "1.2.3.4", "event": "login_fail", "ok": False},
]
solve(logs) -> ["ALERT brute_force src=1.2.3.4 count=5 window=60s"]

logs = [
    {"ts": 0,  "src": "9.9.9.9", "event": "login_fail", "ok": False},
    {"ts": 10, "src": "9.9.9.9", "event": "login_fail", "ok": False},
]
solve(logs) -> []   # only 2 failures — below threshold
```

```
solve([]) -> []     # empty log
```

Algorithm (sliding window)

1. Collect all `login_fail` events and group by `src`. 2. Sort each group by `ts`. 3. For each group, slide a window: for each event at position `i`, count how many events starting at `i` have `ts <= events[i].ts + 59`. 4. If the maximum window count for any source is >= 5, emit its rule string. 5. Return the list sorted lexicographically.

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.5.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List, Dict, Any


def solve(logs: List[Dict[str, Any]]) -> List[str]:
    """Generate detection-rule strings for IPs that exceed the brute-force threshold.

    Args:
        logs: list of dicts with keys ts (int), src (str), event (str), ok (bool).
              Only 'login_fail' events count toward the brute-force rule.
    Returns:
        Sorted list of rule strings of the form:
        'ALERT brute_force src=<IP> count=<N> window=60s'
        One entry per IP that has >= 5 login_fail events within any 60-second window.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() -- see prompt.md")
```

1.5.d · Faster, more accurate threat detection and response

3 pts · auto-graded on the site

Coding problem 1.5.d: Compute SOC incident-response metrics

Skill: Detect Attacks, Collaborate · **Maps to:** 1.5.B · **Language:** Python 3 (pytest)

Problem

A SOC dashboard needs two key metrics to measure how well AI-assisted response is performing:

1. **MTTR** (Mean Time to Respond): the average seconds from detection to resolution, computed across all incidents in the dataset. 2. **auto_resolved**: the count of incidents that were closed automatically by an AI playbook (no human analyst required).

Implement `solve(incidents)` that reads a list of incident records and returns both metrics.

Function contract

Implement `solve(incidents: list[dict]) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`incidents` -- a list of dicts, each with:

- `id` (str): unique incident identifier
- `detected_at` (int): Unix timestamp (seconds) when detected
- `resolved_at` (int): Unix timestamp (seconds) when resolved; always > detected_at
- `severity` (str): one of `"critical"`, `"high"`, `"medium"`, `"low"`
- `auto` (bool): True if auto-resolved, False if human-resolved

Output

A dict with exactly two keys:

- `"mttr"` (float): mean response time in seconds across all incidents
- `"auto_resolved"` (int): count of incidents where `auto == True`

Formula: `mttr = sum(resolved_at - detected_at for each incident) / len(incidents)`

Return `{"mttr": 0.0, "auto_resolved": 0}` for empty input.

Examples

```
incidents = [
    {"id": "INC-001", "detected_at": 1000, "resolved_at": 1120, "severity": "critical", "auto": False},
    {"id": "INC-002", "detected_at": 2000, "resolved_at": 2080, "severity": "high",     "auto": True},
]
# response times: 120s, 80s -> mean = 100.0; auto_resolved = 1
solve(incidents) -> {"mttr": 100.0, "auto_resolved": 1}

solve([]) -> {"mttr": 0.0, "auto_resolved": 0}

incidents = [
    {"id": "INC-003", "detected_at": 0, "resolved_at": 300, "severity": "low", "auto": True},
]
solve(incidents) -> {"mttr": 300.0, "auto_resolved": 1}
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 1.5.d. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List, Dict, Any


def solve(incidents: List[Dict[str, Any]]) -> Dict[str, Any]:
    """Compute MTTR and auto-resolved count for a list of SOC incidents.
```

```
Args:
    incidents: list of dicts with keys:
        id (str), detected_at (int), resolved_at (int),
        severity (str), auto (bool).
Returns:
    {"mttr": float, "auto_resolved": int}
    mttr = mean(resolved_at - detected_at) across all incidents.
    auto_resolved = count of incidents where auto is True.
    Return {"mttr": 0.0, "auto_resolved": 0} for empty input.
"""
# TODO: implement per prompt.md
raise NotImplementedError("implement solve() -- see prompt.md")
```